Lab 5 – Data Visualization I

Tell the story with plotly

Course text for the notebook `lab5_data_visualization_I.py`

dr. Maarten Slembrouck

What is this lab about?

In two labs you turned messy exports into tidy, combined tables. But a coach does not read a table with 111 rows. A good chart shows at a glance what is hidden in the table: who is fastest, who is improving, how hard a training session was.

After this lab you can choose a chart type that fits a question, build the standard charts with `plotly.express`, show extra information with colour, facets and size, and make charts interactive in marimo.

All charts in this text were made with the code above them, on the club's data. In the notebook they are interactive: you can zoom in and hover over a point for more information.

1 The data for this lab

The notebook starts with one cell that repeats the complete cleaning of labs 3 and 4. After that you have two tables:

- `combined`: one row per athlete per test moment, with position, gender, weight, sprint times, jump height and Yo-Yo level;
- `hr_zoned`: the training session per second, with heart rate, speed, name and heart-rate zone.

```
combined.shape, hr_zoned.shape
```

output

```
((111, 15), (12000, 11))
```

Both are **long**: one row per measurement. That is exactly the shape `plotly.express` expects (see lab 4).

The notebook also fixes one colour per position up front, in a dictionary. That way "Forward" has the same colour in every chart, and the reader only has to learn the legend once.

```
POSITION_COLORS = {
    "Forward": "#2a78d6",
    "Midfielder": "#eb6834",
    "Defender": "#1baf7a",
    "Goalkeeper": "#eda100",
}
```

You write a colour as a *hex code*: a hash sign followed by six characters, two for red, two for green and two for blue.

2 Principles: the right chart for the right question

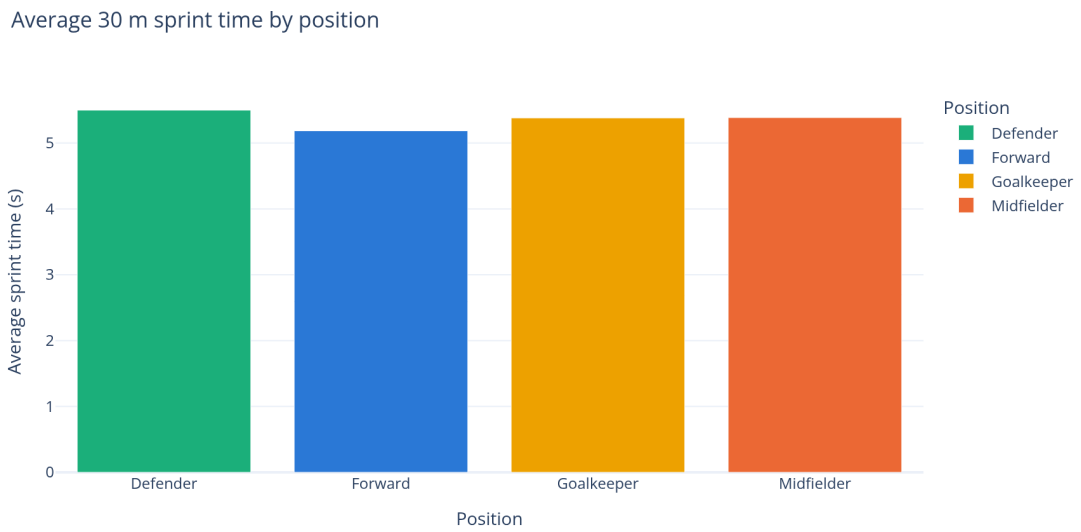
Every chart answers a question. So start from the question, not from the chart type you happen to know.

Kind of question	Example	Suitable chart
Distribution	How are sprint times spread out?	histogram, box plot
Comparison	Which position jumps highest on average?	bar chart, box plot
Relationship	Does a faster sprint mean a higher jump?	scatter plot
Evolution	Is this athlete's heart rate climbing?	line chart

2.1 Honest axes

A chart can mislead as easily as it informs. The best-known trap is a **truncated axis**. Below are the same means twice. In the first chart the y-axis starts at 0:

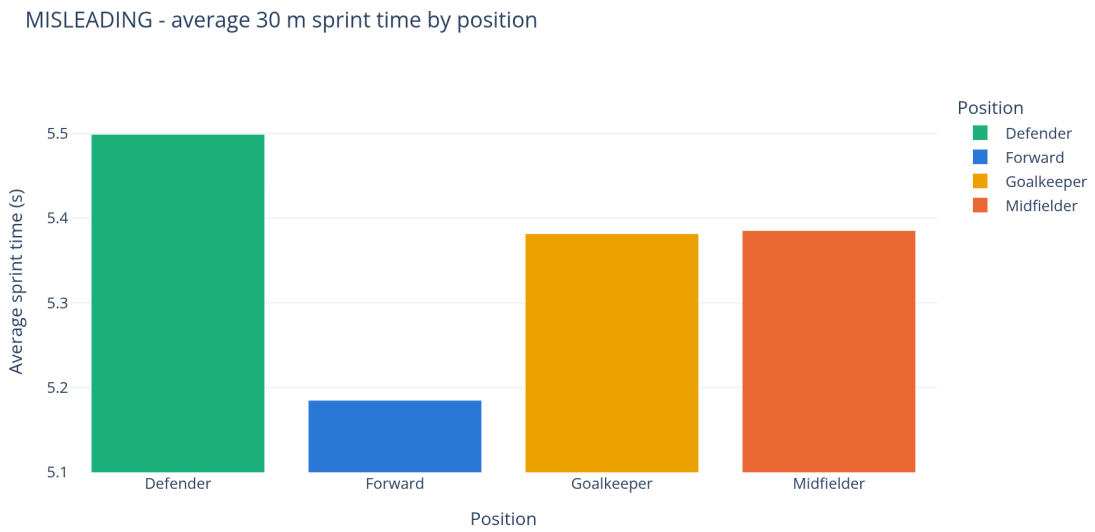
```
sprint_means = combined.groupby("position", as_index=False)["sprint_30m_s"].mean()
fig = px.bar(
    sprint_means, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="Average 30 m sprint time by position",
    labels={"sprint_30m_s": "Average sprint time (s)", "position": "Position"},
)
fig.update_yaxes(rangemode="tozero")
```



`as_index=False` keeps `position` as an ordinary column (instead of making it the index), so you can use it as `x`. In the second chart the y-axis starts just below the smallest value:

```
fig = px.bar(
    sprint_means, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="MISLEADING - average 30 m sprint time by position",
    labels={"sprint_30m_s": "Average sprint time (s)", "position": "Position"},
```

```
)
fig.update_yaxes(range=[5.1, 5.55])
```



The same small differences (at most 0.3 seconds) now look huge. A bar shows a size with its *length*; so that length has to start at zero. For the same reason you never use two different y-axes in one chart: the reader cannot compare the two scales.

3 Making charts with `plotly.express`

`plotly.express` is a library for making interactive charts with little code. You import it as `px`:

```
import plotly.express as px
```

Every function follows the same pattern:

```
fig = px.KIND(table, x="column", y="column", color="column",
              title="...", labels={"column": "Readable name"})
```

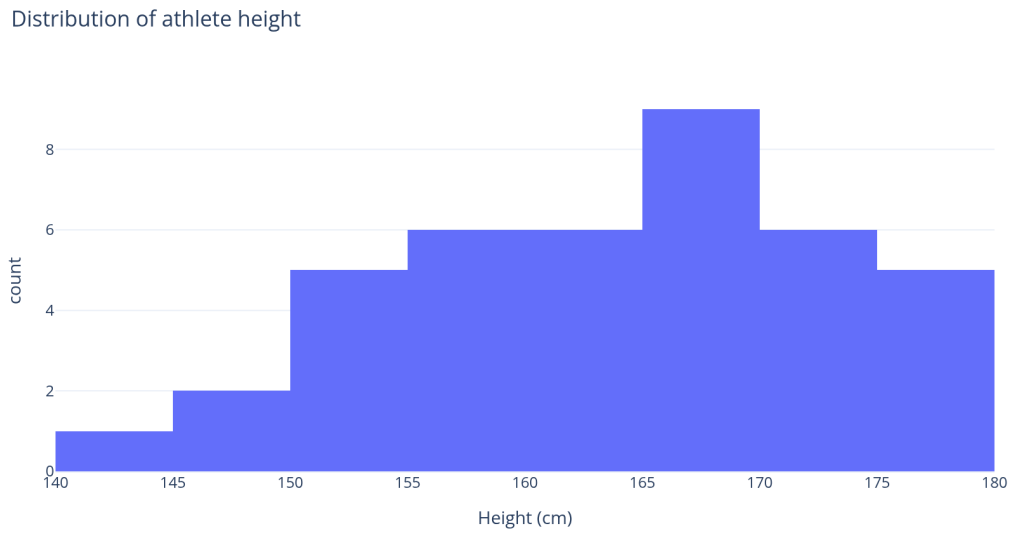
- the **table** comes first; after that you refer to columns by their name, as text;
- `title` is the title above the chart;
- `labels` is a dictionary that translates column names into readable axis labels, with a unit: "Sprint time (s)" instead of `sprint_30m_s`;
- the result is a **figure** object. Put the variable on the last line of a cell and marimo shows the chart.

3.1 Histogram: how is one variable distributed?

A histogram divides the values into bins and shows how many values fall in each bin. `nbins` chooses the (approximate) number of bins.

```
fig = px.histogram(
    athletes, x="height_cm", nbins=15,
    title="Distribution of athlete height",
```

```
    labels={"height_cm": "Height (cm)"},
)
```



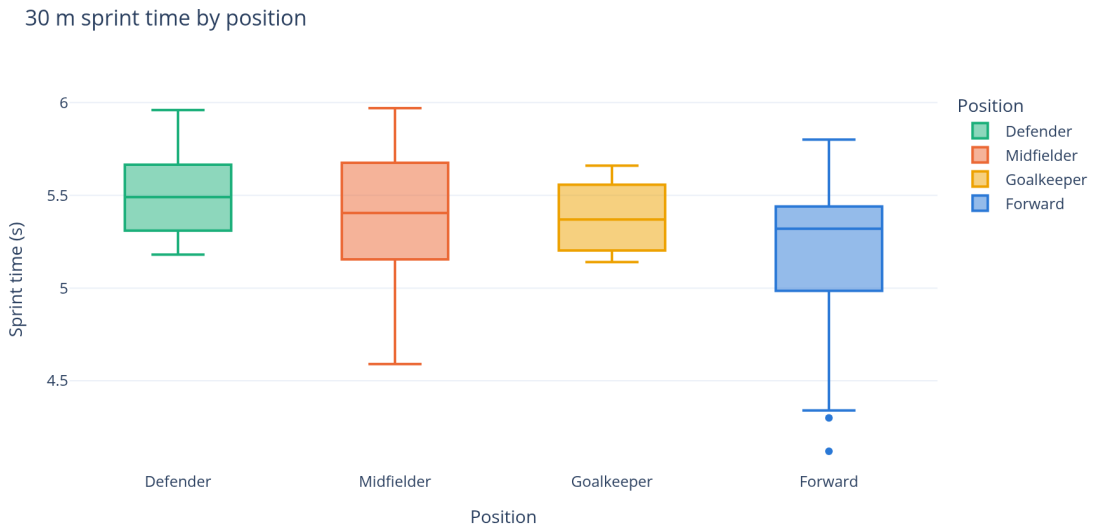
A histogram needs only one column (`x`); the y-axis is the number of athletes per bin, which plotly counts itself. Play with `nbins`: too few bins hide detail, too many make the chart messy.

3.2 Box plot: comparing distributions

A box plot summarizes a distribution in five numbers. This is how you read it:

- the **line in the box** is the median: half of the values are below it, half above;
- the **box** runs from the first to the third quartile: the middle half of the values;
- the **whiskers** run to the lowest and highest value that is not exceptionally far from the box;
- **separate points** are outliers.

```
fig = px.box(
    combined, x="position", y="sprint_30m_s",
    color="position", color_discrete_map=POSITION_COLORS,
    title="30 m sprint time by position",
    labels={"sprint_30m_s": "Sprint time (s)", "position": "Position"},
    hover_data=["name"],
)
```

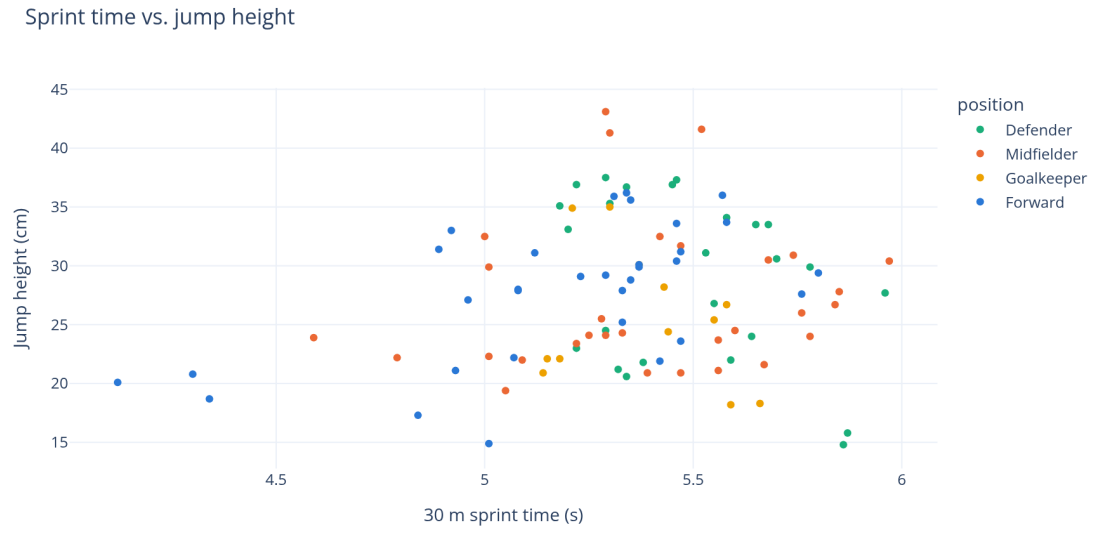


`hover_data=["name"]` adds the name to the information box that appears when you hover over a point. In the notebook you immediately see who that outlier is.

3.3 Scatter plot: is there a relationship?

In a scatter plot every row is a point, with one variable on the x-axis and the other on the y-axis.

```
fig = px.scatter(
    combined, x="sprint_30m_s", y="cmj_height_cm",
    color="position", color_discrete_map=POSITION_COLORS,
    title="Sprint time vs. jump height",
    labels={"sprint_30m_s": "30 m sprint time (s)", "cmj_height_cm": "Jump height
    ↪  (cm)"},
    hover_data=["name", "test_moment"],
)
```



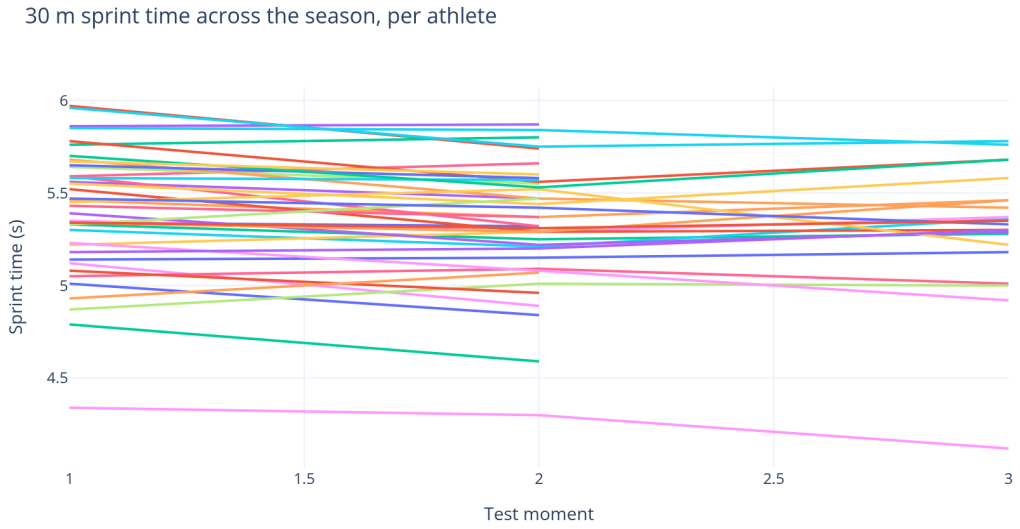
The points form a broad cloud without a clear direction. If there were a strong relationship, they

would lie around a sloping line: going down if faster sprinters (a lower time) also jump higher. That is not the case here: in this data the sprint time says little about the jump height. "No clear relationship" is also an answer to the coach's question. And when you do see a relationship, remember that it is not yet proof that one causes the other.

3.4 Line chart: how does something change?

A line chart connects consecutive points. For that, the data has to be **sorted** on the x-axis, otherwise the line jumps back and forth. With `color="athlete_id"` every athlete gets their own line:

```
fig = px.line(
    combined.sort_values("test_moment"), x="test_moment", y="sprint_30m_s",
    color="athlete_id",
    title="30 m sprint time across the season, per athlete",
    labels={"sprint_30m_s": "Sprint time (s)", "test_moment": "Test moment"},
)
fig.update_layout(showlegend=False)
```



`update_layout(showlegend=False)` hides the legend: with 42 athletes it would take up more space than the chart itself. This works without `pivot_table`, because `combined` is already long.

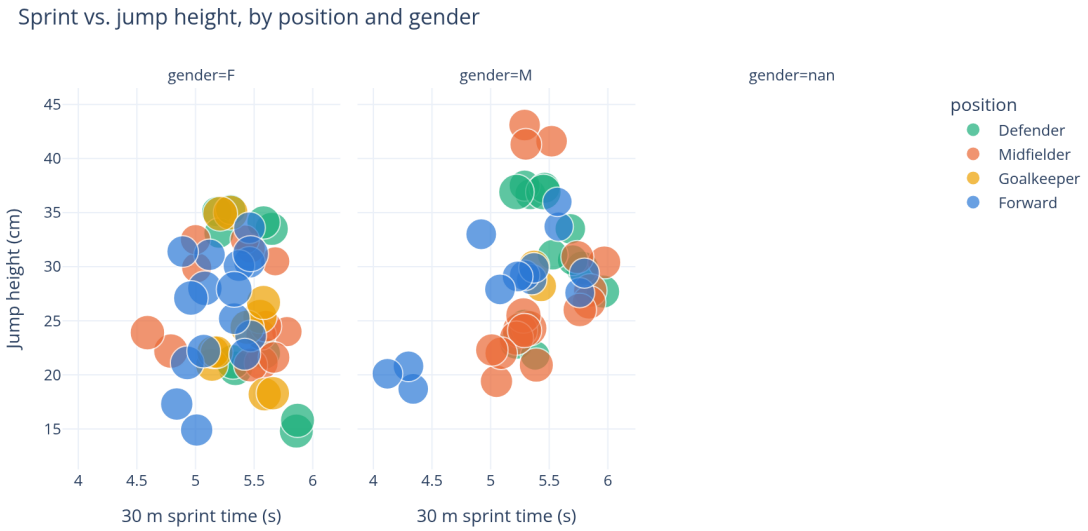
4 More dimensions: colour, facets and size

A scatter plot shows two variables. With three extra arguments you can show more:

- `color="column"`: a category as colour;
- `facet_col="column"`: one small chart per value, side by side, with the same axes;
- `size="column"`: a number as the size of the point.

```
fig = px.scatter(
    combined, x="sprint_30m_s", y="cmj_height_cm",
    color="position", color_discrete_map=POSITION_COLORS,
```

```
    facet_col="gender", size="weight_kg",
    title="Sprint vs. jump height, by position and gender",
    labels={"sprint_30m_s": "30 m sprint time (s)", "cmj_height_cm": "Jump height
    ↪  (cm)"},
)
```



Five variables in one chart: sprint time, jump height, position, gender and weight. Notice the third, empty panel `gender=nan`: a few rows have no gender, and plotly makes a separate panel for them. If you don't want that, drop those rows first with `combined.dropna(subset=["gender"])`, and rename the panel titles with `labels={"gender": "Gender"}`. Facets are often clearer than cramming everything into one chart, because you can compare the panels side by side. But more is not always better: only add a dimension if it helps answer the question.

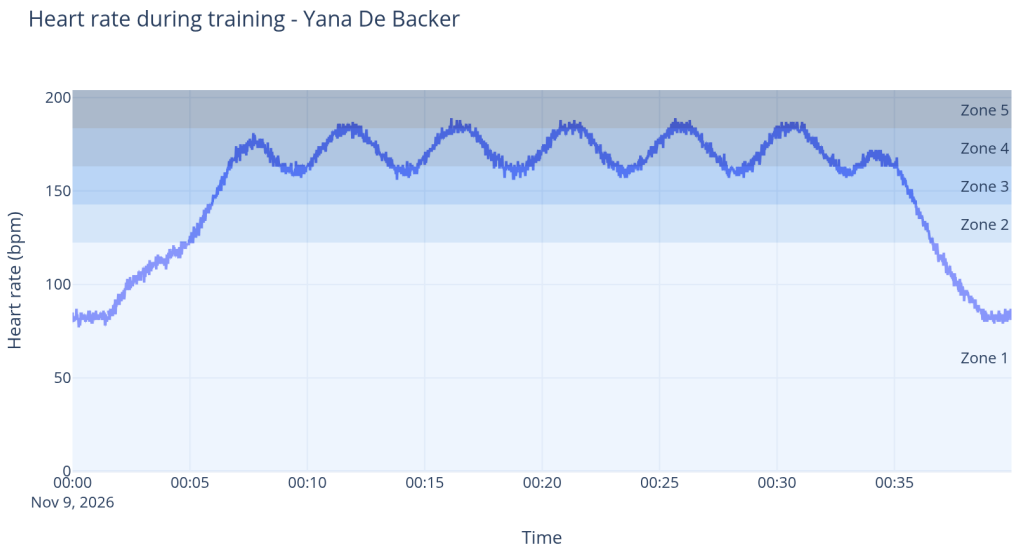
5 Heart rate as a time series, with zones

A line chart of the heart rate becomes much easier to read when you show the heart-rate zones as coloured bands in the background. That is what `add_hrect` is for: a horizontal rectangle between two values on the y-axis.

```
ZONE_COLORS = {"Zone 1": "#cde2fb", "Zone 2": "#86b6ef", "Zone 3": "#3987e5",
               "Zone 4": "#1c5cab", "Zone 5": "#0d366b"}
one_athlete_hr = hr_zoned.loc[hr_zoned["athlete_id"] ==
↪  "A001"].sort_values("timestamp")
hr_max = one_athlete_hr["hr_max"].iloc[0]

fig = px.line(
    one_athlete_hr, x="timestamp", y="heart_rate_bpm",
    title=f"Heart rate during training - {one_athlete_hr['name'].iloc[0]}",
    labels={"heart_rate_bpm": "Heart rate (bpm)", "timestamp": "Time"},
)
zone_edges_pct = [0, 60, 70, 80, 90, 100]
zone_names = ["Zone 1", "Zone 2", "Zone 3", "Zone 4", "Zone 5"]
for lo, hi, zone in zip(zone_edges_pct[:-1], zone_edges_pct[1:], zone_names):
    fig.add_hrect(y0=lo / 100 * hr_max, y1=hi / 100 * hr_max,
                  fillcolor=ZONE_COLORS[zone], opacity=0.35, line_width=0,
```

```
                  annotation_text=zone, annotation_position="right")
fig
```



The loop deserves some explanation:

- `zone_edges_pct[:-1]` is the list without its last element (0, 60, 70, 80, 90): the lower edges. `zone_edges_pct[1:]` is the list without its first element (60, 70, 80, 90, 100): the upper edges.
- `zip(...)` walks through three lists at once and gives one element of each list every time: first `(0, 60, "Zone 1")`, then `(60, 70, "Zone 2")`, and so on.
- `lo / 100 * hr_max` turns a percentage into a heart rate in beats per minute.

In the notebook you use `_lo` instead of `lo`: in marimo a name that starts with `_` stays inside the cell, so it does not clash with variables in other cells.

6 Interactivity in marimo

6.1 UI elements and `.value`

marimo has **UI elements**: dropdowns, sliders, text boxes. A UI element is an ordinary Python variable. Its most important property is `.value`: what the user has chosen right now.

```
athlete_picker = mo.ui.dropdown(
    options=sorted(hr_zoned["name"].unique()),
    value=sorted(hr_zoned["name"].unique())[0],
    label="Athlete",
)
time_window = mo.ui.range_slider(
    start=0, stop=2400, step=60, value=(0, 2400),
    label="Time window (s)", show_value=True,
)
mo.hstack([athlete_picker, time_window])
```

- `options` are the choices; `value` is the initial choice. `.unique()` gives every name once, `sorted` puts them in alphabetical order.
- a `range_slider` has two handles; its `.value` is a tuple, e.g. `(0, 2400)`.
- `mo.hstack([...])` puts elements side by side, `mo.vstack([...])` below each other.

In a *different* cell you use the chosen values to filter the data and make the chart:

```
selected_hr = hr_zoned.loc[
    (hr_zoned["name"] == athlete_picker.value)
    & hr_zoned["time_s"].between(*time_window.value)
].sort_values("timestamp")

reactive_hr_chart = px.line(selected_hr, x="timestamp", y="heart_rate_bpm",
                            title=f"Heart rate - {athlete_picker.value}")
reactive_hr_chart
```

`.between(a, b)` gives a mask that is `True` for values from `a` up to and including `b`. The star in `between(*time_window.value)` *unpacks the tuple*: `between(*(0, 2400))` is the same as `between(0, 2400)`.

Why two cells?

marimo runs a cell again when a variable it depends on changes. When you change the dropdown, `athlete_picker.value` changes, and marimo runs every cell that uses `athlete_picker` again. The cell that *creates* the element must not read its `.value` itself: then the element would be recreated with every choice. So: **create a UI element in one cell, read its value in another**.

6.2 Selecting in the chart (optional)

`mo.ui.plotly(figure)` goes one step further: when you select points in the chart with the box or lasso tool, those points are in `.value`. `pd.DataFrame(selectable_scatter.value)` gives them to you as a table.

7 The mini-dashboard

Today's assignment: a small dashboard for the coach, with **three charts** and **one selector** that at least one chart responds to. The selector (`dashboard_selector`, a choice between the positions) is already in the notebook. You make a list `dashboard_charts` with three figures.

A few tips:

- **Reuse.** The goal is combining, not learning new chart types. Anything from this lab can come back.
- **One question per chart.** First write down, for each chart, which question of the coach it answers, then choose the type (table in section 2).
- **Filter with the selection.** Filter the data with `dashboard_selector.value` before you make the chart, as with the heart-rate chart above.
- **Finishing.** Every chart has a title and readable axis labels with units, and uses `POSITION_COLORS` where positions appear.
- **Layout.** `mo.vstack` and `mo.hstack` arrange the selector and the charts neatly below and next to each other.

8 Summary

Code	What it does
`px.histogram(df, x=, nbins=)`	distribution of one variable
`px.box(df, x=, y=)`	compare distributions per group
`px.scatter(df, x=, y=)`	relationship between two variables
`px.line(df.sort_values(x), x=, y=)`	evolution; sort first
`px.bar(df, x=, y=)`	compare values; axis from 0
`title=`, `labels={...}`, `hover_data=[...]`	title, readable axis labels, extra info on hover
`color=`, `color_discrete_map=`	category as colour, with fixed colours
`facet_col=`, `size=`	panels per value, number as point size
`fig.update_yaxes(rangemode="tozero")`	let the y-axis start at 0
`fig.add_hrect(y0=, y1=, ...)`	horizontal band, e.g. a heart-rate zone
`mo.ui.dropdown`, `mo.ui.range_slider`	dropdown, slider with two handles
`element.value`	the user's current choice
`mo.hstack`, `mo.vstack`	put elements side by side or below each other

Common mistakes

- A bar chart whose y-axis does not start at 0.
- No title, or no units on the axes.
- A line chart on data that is not sorted on the x-axis.
- Creating a UI element and reading its `.value` in the same cell.
- Letting every chart choose its own colours, so "Forward" is blue in one chart and orange in another.

Glossary

figure the object a `px` function returns; marimo shows it as a chart.

median middle value: half of the values are below it, half above.

quartile boundary of the lowest or highest 25 % of the values.

outlier value that lies far from the others.

facet small panel with the same axes, one per value of a column.

UI element dropdown, slider or other element the user chooses something with.